

Имитационное моделирование

Лабораторная работа №6. Реализация модели SIR в подходе сетей Петри

АВТОР

Гашимов Кенан Мухтар оглы

ПРИНАДЛЕЖНОСТЬ

Российский университет дружбы народов

1 Сведения об авторе

- ФИО: Гашимов Кенан Мухтар оглы
- Группа: НКНбд-01-23
- Студенческий билет: 1032235820
- Направление: Математика и компьютерные науки

2 Цель работы

Изучить реализацию модели SIR в аппарате сетей Петри, построить модуль модели и набор вычислительных сценариев на Julia, выполнить детерминированные и стохастические эксперименты, подготовить графики и CSV-таблицы результатов, а также сформировать literate-версии скриптов и производные форматы `clean`, `md`, `ipynb`.

3 Задание

1. Создать рабочий каталог проекта в структуре `DrWatson`.
2. Установить зависимости для моделирования, визуализации и документирования.
3. Реализовать модуль `SIRPetri.jl` с описанием сети Петри и функциями симуляции.
4. Выполнить базовый прогон модели и сохранить результаты в `CSV` и `PNG`.
5. Провести параметрическое исследование по коэффициенту заражения β .
6. Построить GIF-анимацию детерминированной динамики.
7. Подготовить итоговый сравнительный отчёт по ранее сохранённым данным.
8. Для каждого сценария получить `clean`, `md` и `ipynb` представления при помощи `Literate.jl`.
9. Интегрировать полученные материалы в итоговый отчёт и презентацию.

4 Теоретическое введение

4.1 Сети Петри и модель SIR

Сеть Петри представляет собой ориентированный двудольный граф, в котором позиции задают состояния системы, переходы описывают события, дуги определяют разрешённые связи, а маркировка показывает текущее распределение фишек по позициям [1]. Такой аппарат удобен для моделирования дискретных событий и конкурирующих процессов.

В данной лабораторной работе рассматривается эпидемическая модель SIR с тремя состояниями:

- **S** — восприимчивые;
- **I** — инфицированные;
- **R** — выздоровевшие.

Используются два перехода:

- `infection: S + I -> I + I`;
- `recovery: I -> R`.

Тем самым заражение интерпретируется как совместное наличие восприимчивого и инфицированного, а выздоровление — как уход токена из состояния **I** в состояние **R**.

4.2 Детерминированная и стохастическая формы модели

Для непрерывной аппроксимации динамики используется система обыкновенных дифференциальных уравнений:

$$\frac{dS}{dt} = -\beta SI, \quad \frac{dI}{dt} = \beta SI - \gamma I, \quad \frac{dR}{dt} = \gamma I.$$

Эта форма даёт усреднённую гладкую траекторию по закону действующих масс. Стохастическая форма строится по алгоритму Гиллеспи [2], где на каждом шаге вычисляются интенсивности событий `infection` и `recovery`, после чего случайно выбираются время следующего события и тип перехода.

В текущей реализации интенсивность заражения задаётся как `β * S * I` без нормировки на размер популяции. Именно это объясняет очень быстрый рост **I** и почти мгновенное истощение **S** в полученных экспериментах.

4.3 Инструменты работы

В работе использовались:

- `Julia` как язык реализации [3];
- `DrWatson` для организации воспроизводимой структуры проекта [4];
- `OrdinaryDiffEq` для численного решения системы ОДУ;
- `Plots` для построения графиков и анимации;
- `CSV` и `DataFrames` для сохранения и анализа таблиц результатов;
- `Literate.jl` для генерации `clean`, `md` и `ipynb` представлений [5].

4.4 Литературное программирование

Литературное программирование рассматривает программу как связный документ, в котором код и объяснение образуют единое целое [6]. Такой подход особенно удобен в

учебной лабораторной работе, поскольку позволяет из одного исходного скрипта получить:

- исполняемую **clean**-версию;
- Markdown-документацию;
- Jupyter notebook.

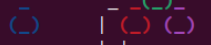
Воспроизводимые вычислительные документы такого типа естественно вписываются в практику исследовательской работы [7].

5 Выполнение лабораторной работы

5.1 Подготовка окружения

Сначала была запущена рабочая сессия Julia, после чего для проекта была использована стандартная схема **DrWatson**: создание каталога проекта, подключение пакета и активация окружения.

```
kenanguka@kenanguka:~/simmod/labs/lab06$ julia
```



```
| Documentation: https://docs.julialang.org  
|  
| Type "?" for help, "]"? for Pkg help.  
|  
| Version 1.12.5 (2026-02-09)  
| Official https://julialang.org release
```

Запуск Julia REPL

На скриншоте показан старт рабочей сессии Julia, с которой начиналась вся настройка проекта и последующее выполнение экспериментов.

```
1 julia> using DrWatson
```

Подключение пакета DrWatson

После запуска интерпретатора был подключён пакет `DrWatson`, который обеспечивает типовую структуру проекта и удобные функции `srcdir`, `datadir`, `plotsdir`.

```
julia> initialize_project("lab_06_SIR_petri"; authors="Idrisov Jafer")
Activating new project at `~/imit-model/2026-1--study--simulation-modeling/labs/lab06/lab_06_SIR_petri`
Updating registry at `~/.julia/registries/General.toml`
Resolving package versions...
Updating `~/imit-model/2026-1--study--simulation-modeling/labs/lab06/lab_06_SIR_petri/Project.toml`
```

Инициализация проекта lab 06 SIR petri

Здесь показано создание проекта `lab_06_SIR_petri` через `initialize_project`. На этом шаге были сформированы каталоги `src`, `scripts`, `data`, `plots`, `test` и служебные файлы `Project.toml` и `Manifest.toml`.

```
julia> @quickactivate
Activating project at `~/imit-model/2026-1--study--simulation-modeling/labs/lab06/lab_06_SIR_petri`
```

Активация окружения проекта

После инициализации проекта было выполнено `@quickactivate`, что связало текущую REPL-сессию именно с каталогом лабораторной работы.

```
b06/lab_06_SIR_petri
```

```
julia> using Pkg
julia> Pkg.add([
    "DrWatson",
    "AlgebraicPetri",
    "Catlab",
    "OrdinaryDiffEq",
    "Plots",
    "DataFrames",
    "CSV",
    "Literate",
    "IJulia",
    "Quarto",
])
```

Установка зависимостей проекта

На этом шаге были установлены основные зависимости: `AlgebraicPetri`, `Catlab`, `OrdinaryDiffEq`, `Plots`, `CSV`, `DataFrames`, `Literate`, `IJulia`, `Quarto`.

```
[05823500] + OpenLibm_jll v0.8.7+0
[efcefd7] + PCRE2_jll v10.44.0+1
[bea87d4a] + SuiteSparse_jll v7.8.3+2
[8e850b90] + libblastrampoline_jll v5.15.0+0
Info Packages marked with ^ and ~ have new versions available. Those with ^ may be upgradable, but those with ~ are restricted by compatibility constraints from upgrading. To see why use `status --outdated -m`
Precompiling packages finished.
173 dependencies successfully precompiled in 525 seconds. 274 already precompiled.
```

Завершение установки пакетов

Скриншот фиксирует окончание `precompile` и готовность окружения к выполнению кода лабораторной работы.

5.2 Структура файлов лабораторной работы

По итогам выполнения работы были подготовлены следующие основные файлы:

- `src/SIRPetri.jl` — модуль модели;
- `scripts/sirpetri_run.jl` — базовый прогон;
- `scripts/sirpetri_scan_parameters.jl` — параметрическое исследование по β ;
- `scripts/sirpetri_animate.jl` — построение GIF-анимации;
- `scripts/sirpetri_report.jl` — итоговые сравнительные графики;
- `docs/*.md` — Markdown-представления сценариев;
- `notebook/*.ipynb` — notebook-представления;
- `data/*.csv` и `plots/*.png/*.gif` — результаты вычислений.

5.3 Модуль `src/SIRPetri.jl`

Модуль `SIRPetri.jl` содержит:

- построение размеченной сети Петри;
- функцию правой части ОДУ;
- детерминированную симуляцию;
- стохастическую симуляцию по алгоритму Гиллеспи;
- построение графиков;
- вывод объекта Graphviz для визуализации сети.

5.3.1 Ключевые фрагменты кода модуля

```
function build_sir_network(beta::Real = 0.3, gamma::Real = 0.1)
    states = [:S, :I, :R]
    net = LabelledPetriNet(
        states,
        :infection => ([:S, :I] => [:I, :I]),
        :recovery => ([:I] => [:R]),
    )

    u0 = [990.0, 10.0, 0.0]
    return net, u0, states
end
```

Этот фрагмент задаёт структуру сети Петри с тремя состояниями и двумя переходами. Начальная маркировка фиксирует популяцию $990 + 10 + 0 = 1000$.

```
function sir_ode(net, rates::AbstractVector{<:Real} = [0.3, 0.1])
    function f!(du, u, p, t)
        s, i, r = u
        beta, gamma = rates

        infection_rate = beta * s * i
        recovery_rate = gamma * i

        du[1] = -infection_rate
        du[2] = infection_rate - recovery_rate
        du[3] = recovery_rate
        return nothing
    end

    return f!
end
```

Здесь реализована детерминированная форма модели. Из-за множителя $S * I$ без нормировки скорость заражения при начальных $S = 990$, $I = 10$ оказывается очень большой уже с первых шагов интегрирования.

```
function simulate_stochastic(net, u0, tspan; rates = [0.3, 0.1], rng = Random.GLOBAL_RNG)
    u = Int.(round.(u0))
    t = Float64(tspan[1])
```

```

tmax = Float64(tspan[2])
beta, gamma = rates

while t < tmax
    s, i, r = u
    a_inf = beta * s * i
    a_rec = gamma * i
    a0 = a_inf + a_rec
    if a0 <= 0
        break
    end
    dt = -log(rand(rng)) / a0
    event_draw = rand(rng) * a0
    if event_draw < a_inf
        u[1] -= 1
        u[2] += 1
    else
        u[2] -= 1
        u[3] += 1
    end
    t += dt
end
end

```

В стохастической части модели на каждом шаге рассчитываются пропускные способности заражения и выздоровления. Это даёт ступенчатую траекторию с дискретными событиями.

5.4 Базовый прогон: `scripts/sirpetri_run.jl`

Скрипт `sirpetri_run.jl` запускает два сценария:

- детерминированную симуляцию;
- стохастическую симуляцию с фиксированным зерном генератора случайных чисел.

Он сохраняет результаты в `sir_det.csv` и `sir_stoch.csv`, а также строит графики `sir_det_dynamics.png` и `sir_stoch_dynamics.png`.

5.4.1 Ключевой фрагмент кода

```

function main()
    beta = 0.3
    gamma = 0.1
    tmax = 100.0

    net, u0, _ = SIRPetri.build_sir_network(beta, gamma)

    df_det = SIRPetri.simulate_deterministic(
        net, u0, (0.0, tmax); saveat = 0.5, rates = [beta, gamma]
    )
    CSV.write(datadir("sir_det.csv"), df_det)

```

```

Random.seed!(123)
df_stoch = SIRPetri.simulate_stochastic(
    net, u0, (0.0, tmax); rates = [beta, gamma]
)
CSV.write(datadir("sir_stoch.csv"), df_stoch)
end

```

Этот фрагмент показывает общую схему базового эксперимента: построение сети, запуск двух видов симуляции и сохранение результатов.

```

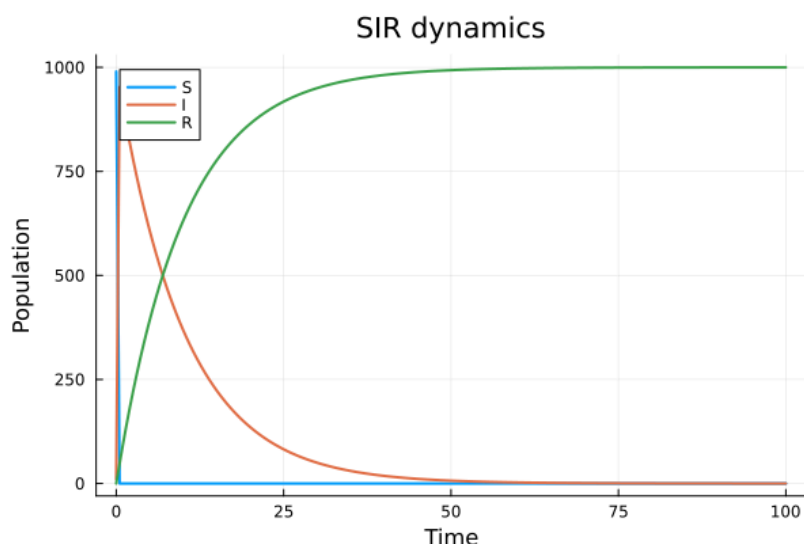
julia> include("scripts/sirpetri_run.jl")
Could not create decoration from factory! Running with no decorations.
Baseline SIR runs completed.

```

Запуск базового сценария `sirpetri_run.jl`

Скриншот подтверждает успешное выполнение скрипта `sirpetri_run.jl` и завершение базовых экспериментов.

5.4.2 График `sir_det_dynamics.png`

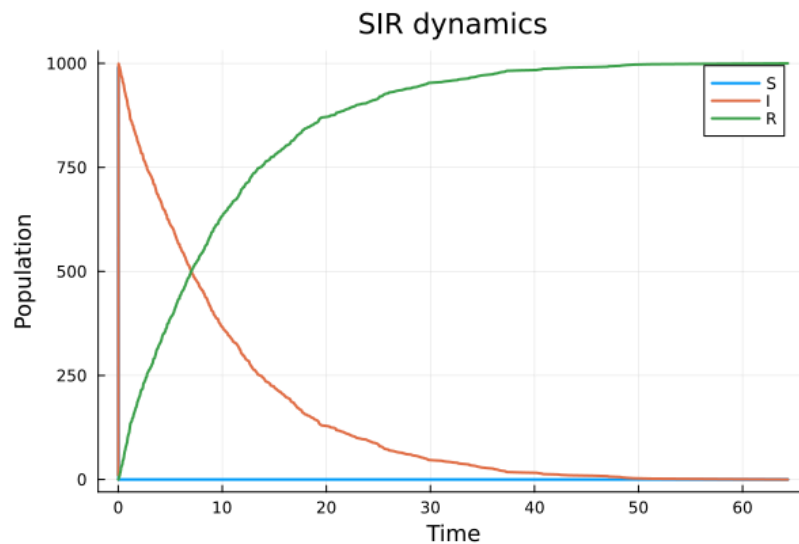


Детерминированная динамика SIR

На детерминированном графике видно, что число восприимчивых практически мгновенно падает к нулю, а число инфицированных достигает максимума уже при $t = 0.5$. Максимум составляет примерно $I_{\max} = 952.69$, после чего $I(t)$ монотонно убывает, а $R(t)$ растёт до значения 999.95 к $t = 100$.

Таким образом, в текущей реализации модель даёт очень жёсткую вспышку: почти вся популяция за короткое время переходит в состояние заражения, а затем плавно выздоравливает.

5.4.3 График `sir_stoch_dynamics.png`



Стохастическая динамика SIR

Стохастическая траектория имеет ступенчатый характер. В начале моделирования наблюдается серия быстрых актов заражения, поэтому $I(t)$ почти мгновенно возрастает до 999 уже при $t \approx 0.037$. Затем начинается медленный каскад выздоровлений, и к $t \approx 64.35$ система приходит в состояние $S = 0$, $I = 0$, $R = 1000$.

Этот график хорошо показывает отличие событийной стохастической модели от гладкой детерминированной аппроксимации.

5.4.4 CSV-таблица `sir_det.csv`

	time	S	I	R
1	0.0	990.0	10.0	0.0
2	0.5	2.470530176355568e-7	952.6913722338431	47.30862751910418
3	1.0	3.2825156170551274e-7	906.2280656435082	93.7719340282407
4	1.5	-1.7836497094175214e-7	862.0308018392616	137.96919833910425
5	2.0	-2.659696124153069e-7	819.9890636317962	180.01093663417439
6	2.5	2.3889984720578033e-7	779.9977246032496	220.00227515785144
7	3.0	-3.2351753804476944e-7	741.9567872372246	258.043213086294
8	3.5	-2.3068224861046615e-7	705.7711276509772	294.22887257970586
9	4.0	3.040277104817814e-8	671.3502633347057	328.6497366348925
10	4.5	3.63345392125534e-8	638.6081246228556	361.3918753408108
11	5.0	3.5473303030046637e-7	607.4630303550000	392.537161000746

Содержимое файла `sir_det.csv`

Файл `sir_det.csv` содержит 201 строку данных и 4 столбца:

- `time` — время;
- `S` — число восприимчивых;
- `I` — число инфицированных;
- `R` — число выздоровевших.

По первым строкам таблицы видно, что уже к $t = 0.5$ компонент `S` практически зануляется, а `I` становится порядка 952.69. В последних строках `I` убывает до 0.045, а `R`

выходит на плато около 999.95 .

5.4.5 CSV-таблица `sir_stoch.csv`

	time	S	I	R
1	0.0	990	10	0
2	0.00021931839962565963	989	11	0
3	0.0002547101547375829	988	12	0
4	0.00043545667342433645	987	13	0
5	0.00124186225244802	986	14	0
6	0.0013731233337092817	985	15	0
7	0.0015175893857365716	984	16	0
8	0.0021941207102495813	983	17	0
9	0.0023963777301024003	982	18	0
10	0.0024582790296121867	981	19	0

Содержимое файла `sir_stoch.csv`

Файл `sir_stoch.csv` содержит 1991 строку данных и те же столбцы `time`, `S`, `I`, `R`, но все состояния меняются дискретно. В первых строках хорошо видны последовательные события заражения: `S` уменьшается на единицу, а `I` увеличивается на единицу. В последних строках происходит убывание `I` до нуля и рост `R` до 1000 .

Эта таблица даёт полную событийную траекторию стохастической модели.

5.5 Подготовка `literate`-представлений базового сценария

После успешного прогона был подключён пакет `Literate.jl` и выполнена генерация производных форматов.

```
julia> using Literate
```

Подключение пакета `Literate`

Скриншот показывает подключение `Literate.jl`, который использовался для построения `clean`, `md` и `ipynb` представлений каждого сценария.

```
julia> Literate.script("scripts/sirpetri_run.jl", "scripts"; name="sirpetri_run_clean")
[ Info: generating plain script file from `~/imit-model/2026-1--study--simulation-modeling/labs/lab06/lab_06_SIR_petri/scripts/sirpetri_run.jl`
[ Info: writing result to `~/imit-model/2026-1--study--simulation-modeling/labs/lab06/lab_06_SIR_petri/scripts/sirpetri_run_clean.jl`
"/home/daidrisov/imit-model/2026-1--study--simulation-modeling/labs/lab06/lab_06_SIR_petri/scripts/sirpetri_run_clean.jl"
```

Clean-версия скрипта `sirpetri_run.jl`

Clean -представление содержит только исполняемый код без текстовой literate-разметки.

```
julia> Literate.markdown("scripts/sirpetri_run.jl", "docs"; name="sirpetri_run", execute=true)
[ Info: generating markdown page from `~/imit-model/2026-1--study--simulation-modeling/labs/lab06/lab_06_SIR_petri/scripts/sirpetri_run.jl`
[ Info: writing result to `~/imit-model/2026-1--study--simulation-modeling/labs/lab06/lab_06_SIR_petri/docs/sirpetri_run.md`
"/home/daidrisov/imit-model/2026-1--study--simulation-modeling/labs/lab06/lab_06_SIR_petri/docs/sirpetri_run.md"
```

Markdown-версия сценария `sirpetri_run`

Markdown-файл выступает в роли промежуточной документации по базовому прогону.

```
julia> Literate.notebook("scripts/sirpetri_run.jl", "notebook"; name="sirpetri_run", execute=true)
[ Info: generating notebook from `~/imit-model/2026-1--study--simulation-modeling/labs/lab06/lab_06_SIR_petri/scripts/sirpetri_run.jl`
[ Info: executing notebook `sirpetri_run.ipynb`
[ Info: writing result to `~/imit-model/2026-1--study--simulation-modeling/labs/lab06/lab_06_SIR_petri/notebook/sirpetri_run.ipynb`
"/home/daidrisov/imit-model/2026-1--study--simulation-modeling/labs/lab06/lab_06_SIR_petri/notebook/sirpetri_run.ipynb"
```

Notebook-версия сценария `sirpetri_run`

Notebook-версия позволяет воспроизводить тот же эксперимент интерактивно по ячейкам.

5.6 Параметрическое исследование:

`scripts/sirpetri_scan_parameters.jl`

Скрипт `sirpetri_scan_parameters.jl` выполняет серию детерминированных прогонов для β в диапазоне `0.1:0.05:0.8` при фиксированном $\gamma = 0.1$, после чего сохраняет таблицу `sir_scan.csv` и график `sir_scan.png`.

5.6.1 Ключевой фрагмент кода

```
function main()
    beta_range = 0.1:0.05:0.8
    gamma_fixed = 0.1
    tmax = 100.0

    results = DataFrame(beta = Float64[], peak_I = Float64[], final_R = Float64[])

    for beta in beta_range
        net, u0, _ = SIRPetri.build_sir_network(beta, gamma_fixed)
        df = SIRPetri.simulate_deterministic(
```

```

        net, u0, (0.0, tmax); saveat = 0.5, rates = [beta, gamma_fixed]
    )
    push!(results, (beta = Float64(beta), peak_I = maximum(df.I), final_R = df.f
end

CSV.write(datadir("sir_scan.csv"), results)
end

```

Здесь центральным элементом является цикл по значениям β , который для каждого прогона вычисляет максимум I и итоговое значение R .

```

julia> include("scripts/sirpetri_scan_parameters.jl")
jl Parameter scan completed.

```

Запуск скрипта параметрического исследования

Скриншот подтверждает выполнение `sirpetri_scan_parameters.jl` и формирование итоговой таблицы и графика.

5.6.2 График `sir_scan.png`

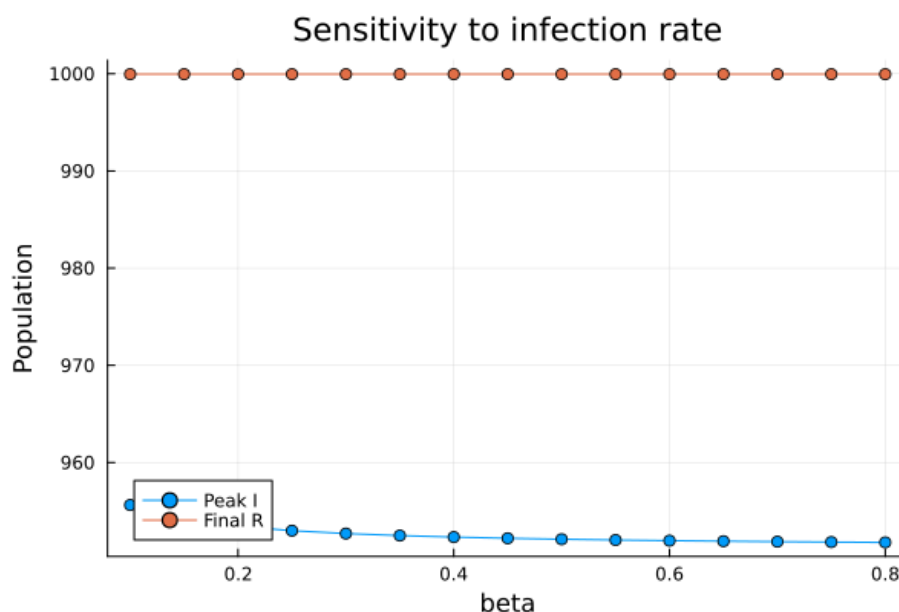


График зависимости от коэффициента заражения

График содержит две кривые:

- `Peak I( $\beta$ )` — максимальное число инфицированных;
- `Final R( $\beta$ )` — итоговое число выздоровевших.

В полученных данных обе зависимости оказываются очень слабо изменяющимися: `peak_I` лежит в диапазоне от 951.78 до 955.63, а `final_R` — около 999.9545. При этом максимум I даже слегка уменьшается с ростом β . Такое поведение связано не с типичной эпидемиологической динамикой, а с тем, что заражение в текущей реализации и без того происходит почти мгновенно для всех рассмотренных значений параметра.

5.6.3 CSV-таблица `sir_scan.csv`

	beta	peak_I	final_R
1	0.1	955.625967117857	999.9543789676285
2	0.15	954.1568964325731	999.9544516223195
3	0.2	953.4236812173239	999.9544864197777
4	0.25	952.9841976491601	999.954509713774
5	0.3	952.6913722338431	999.9545235233431
6	0.35	952.4823042281093	999.9545352234009
7	0.4	952.3255540093855	999.9545430130853
8	0.45	952.2036643215422	999.9545482376027
9	0.5	952.1061667265205	999.9545540811765
10	0.55	952.0264101284754	999.9545584068065
11	0.6	951.9599529308231	999.9545617869207
12	0.65	951.9037269561279	999.9545635678068
13	0.7	951.8555405370909	999.9545665066365
14	0.75	951.813781235434	999.9545693681596
15	0.8	951.7772424502728	999.9545703252896

Содержимое файла `sir_scan.csv`

Таблица `sir_scan.csv` содержит 15 строк данных со столбцами:

- `beta` — коэффициент заражения;
- `peak_I` — максимальное число инфицированных;
- `final_R` — итоговое число выздоровевших.

Минимум `peak_I` достигается при $\beta = 0.8$ и равен `951.7772`, максимум — при $\beta = 0.1$ и равен `955.6260`. Значения `final_R` отличаются только в четвёртом знаке после запятой, что подтверждает слабую чувствительность текущей реализации к изменению β .

5.6.4 Производные форматы параметрического сценария

```
julia> Literate.script("scripts/sirpetri_scan_parameters.jl", "scripts"; name="s
irpetri_scan_parameters_clean")
[ Info: generating plain script file from `~/imit-model/2026-1--study--simulatio
n-modeling/labs/lab06/lab_06_SIR_petri/scripts/sirpetri_scan_parameters.jl`
[ Info: writing result to `~/imit-model/2026-1--study--simulation-modeling/labs/
lab06/lab_06_SIR_petri/scripts/sirpetri_scan_parameters_clean.jl`
"/home/daidrisov/imit-model/2026-1--study--simulation-modeling/labs/lab06/lab_06
_SIR_petri/scripts/sirpetri_scan_parameters_clean.jl"
```

Clean-версия `sirpetri_scan_parameters`

Этот файл содержит исполняемый вариант параметрического скрипта без поясняющего текста.

```
julia> Literate.markdown("scripts/sirpetri_scan_parameters.jl", "docs"; name="sirpetri_scan_parameters", execute=true)
[ Info: generating markdown page from `~/imit-model/2026-1--study--simulation-modeling/labs/lab06/lab_06_SIR_petri/scripts/sirpetri_scan_parameters.jl`
[ Info: writing result to `~/imit-model/2026-1--study--simulation-modeling/labs/lab06/lab_06_SIR_petri/docs/sirpetri_scan_parameters.md`
"/home/daidrisov/imit-model/2026-1--study--simulation-modeling/labs/lab06/lab_06_SIR_petri/docs/sirpetri_scan_parameters.md"
```

Markdown-представление параметрического сценария

Markdown-файл фиксирует ход параметрического эксперимента и служит текстовым артефактом literate-подхода.

```
julia> Literate.notebook("scripts/sirpetri_scan_parameters.jl", "notebook"; name="sirpetri_scan_parameters", execute=true)
[ Info: generating notebook from `~/imit-model/2026-1--study--simulation-modeling/labs/lab06/lab_06_SIR_petri/scripts/sirpetri_scan_parameters.jl`
[ Info: executing notebook `sirpetri_scan_parameters.ipynb`
[ Info: writing result to `~/imit-model/2026-1--study--simulation-modeling/labs/lab06/lab_06_SIR_petri/notebook/sirpetri_scan_parameters.ipynb`
"/home/daidrisov/imit-model/2026-1--study--simulation-modeling/labs/lab06/lab_06_SIR_petri/notebook/sirpetri_scan_parameters.ipynb"
```

Notebook-представление параметрического сценария

Notebook-версия удобна для повторного запуска серии экспериментов в интерактивной среде.

5.7 Анимация: `scripts/sirpetri_animate.jl`

Скрипт `sirpetri_animate.jl` строит GIF-анимацию по детерминированной траектории, в которой на каждом кадре отображается текущее число токенов в состояниях `S`, `I`, `R`.

5.7.1 Ключевой фрагмент кода

```
function main()
    beta = 0.3
    gamma = 0.1
    tmax = 100.0

    net, u0, _ = SIRPetri.build_sir_network(beta, gamma)
    df = SIRPetri.simulate_deterministic(
        net, u0, (0.0, tmax); saveat = 0.2, rates = [beta, gamma]
    )

    anim = @animate for i in 1:nrow(df)
        bar(["S", "I", "R"], [df.S[i], df.I[i], df.R[i]];
            ylim = (0, 1000), legend = false)
    end

    gif(anim, plotsdir("sir_animation.gif"), fps = 10)
end
```

Этот фрагмент показывает, что анимация строится напрямую по строкам таблицы состояния.

```
julia> include("scripts/sirpetri_animate.jl")
Could not create decoration from factory! Running with no decorations.
[ Info: Saved animation to /home/daidrisov/imit-model/2026-1--study--simulation-modeling/labs/lab06/lab_06_SIR_petri/plots/sir_animation.gif
Animation saved.
```

Запуск сценария анимации

Скриншот фиксирует запуск `sirpetri_animate.jl` и сохранение файла `sir_animation.gif`.

Логика анимации заключается в том, что на первых кадрах быстро уменьшается **S**, затем почти весь столбик переходит в **I**, а далее начинается медленный рост **R** и спад **I**.

5.7.2 Производные форматы скрипта анимации

```
julia> Literate.script("scripts/sirpetri_animate.jl", "scripts"; name="sirpetri_animate_clean")
[ Info: generating plain script file from `~/imit-model/2026-1--study--simulation-modeling/labs/lab06/lab_06_SIR_petri/scripts/sirpetri_animate.jl`
[ Info: writing result to `~/imit-model/2026-1--study--simulation-modeling/labs/lab06/lab_06_SIR_petri/scripts/sirpetri_animate_clean.jl`
"/home/daidrisov/imit-model/2026-1--study--simulation-modeling/labs/lab06/lab_06_SIR_petri/scripts/sirpetri_animate_clean.jl"
```

Clean-версия скрипта анимации

Clean -представление анимации содержит конечный исполняемый код построения GIF.

```
julia> Literate.markdown("scripts/sirpetri_animate.jl", "docs"; name="sirpetri_animate", execute=true)
[ Info: generating markdown page from `~/imit-model/2026-1--study--simulation-modeling/labs/lab06/lab_06_SIR_petri/scripts/sirpetri_animate.jl`
[ Info: writing result to `~/imit-model/2026-1--study--simulation-modeling/labs/lab06/lab_06_SIR_petri/docs/sirpetri_animate.md`
"/home/daidrisov/imit-model/2026-1--study--simulation-modeling/labs/lab06/lab_06_SIR_petri/docs/sirpetri_animate.md"
```

Markdown-представление скрипта анимации

Markdown-версия фиксирует пояснения к построению кадров и сохранению анимации.

```
julia> Literate.notebook("scripts/sirpetri_animate.jl", "notebook"; name="sirpetri_animate", execute=true)
[ Info: generating notebook from `~/imit-model/2026-1--study--simulation-modeling/labs/lab06/lab_06_SIR_petri/scripts/sirpetri_animate.jl`
[ Info: executing notebook `sirpetri_animate.ipynb`
[ Info: writing result to `~/imit-model/2026-1--study--simulation-modeling/labs/lab06/lab_06_SIR_petri/notebook/sirpetri_animate.ipynb`
"/home/daidrisov/imit-model/2026-1--study--simulation-modeling/labs/lab06/lab_06_SIR_petri/notebook/sirpetri_animate.ipynb"
```

Notebook-представление скрипта анимации

Notebook-версия позволяет последовательно воспроизводить процесс создания анимации в интерактивной форме.

5.8 Итоговый отчётный сценарий:

`scripts/sirpetri_report.jl`

Скрипт `sirpetri_report.jl` не моделирует систему заново, а загружает ранее сохранённые данные и строит итоговые графики `comparison.png` и `sensitivity.png`.

5.8.1 Ключевой фрагмент кода

```
function main()
    df_det = CSV.read(datadir("sir_det.csv"), DataFrame)
    df_stoch = CSV.read(datadir("sir_stoch.csv"), DataFrame)
    df_scan = CSV.read(datadir("sir_scan.csv"), DataFrame)

    n = min(nrow(df_det), nrow(df_stoch))

    p1 = plot(df_det.time[1:n], df_det.I[1:n], label = "Deterministic I")
    plot!(p1, df_stoch.time[1:n], df_stoch.I[1:n], label = "Stochastic I")
    savefig(p1, plotsdir("comparison.png"))

    p2 = plot(df_scan.beta, df_scan.peak_I, marker = :circle, label = "Peak I")
    savefig(p2, plotsdir("sensitivity.png"))
end
```

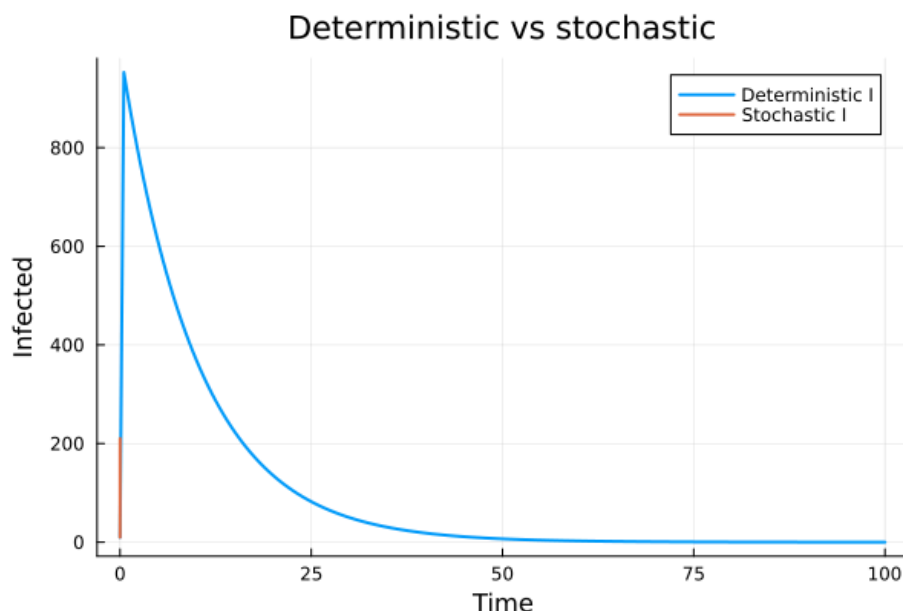
Итоговый сценарий объединяет результаты базового прогона и параметрического анализа в отдельные отчётные рисунки.

```
julia> include("scripts/sirpetri_report.jl")
Could not create decoration from factory! Running with no decorations.
Report plots saved.
```

Запуск итогового отчётного сценария

Скриншот подтверждает выполнение `sirpetri_report.jl` и сохранение итоговых графиков.

5.8.2 График `comparison.png`



Сравнение детерминированной и стохастической траекторий

На этом графике совместно показаны кривые $I(t)$ для двух режимов моделирования. Стохастическая траектория стартует резче и быстрее достигает пика, тогда как детерминированная аппроксимация остаётся гладкой. Обе кривые отражают одну и ту же общую картину: резкий всплеск числа инфицированных и последующее затухание.

5.8.3 График `sensitivity.png`

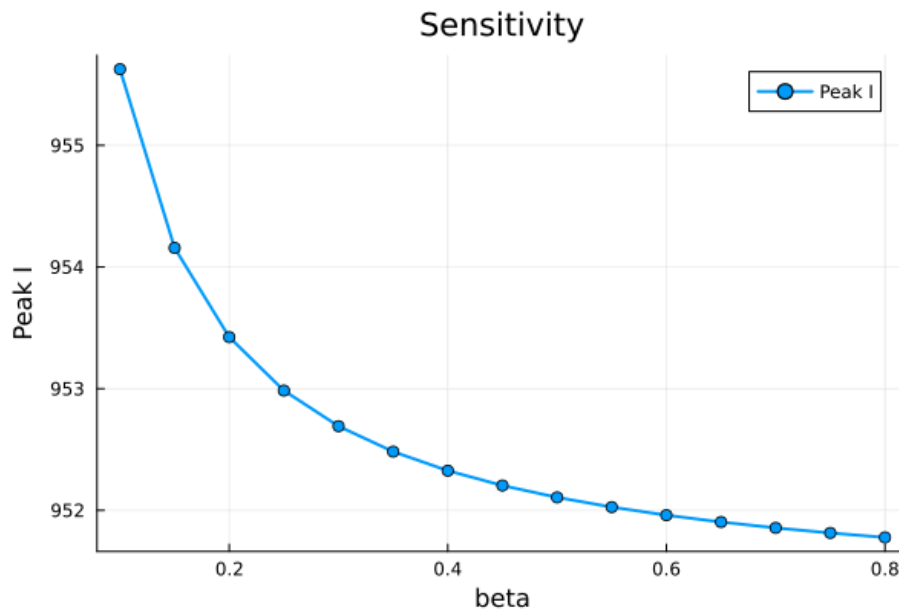


График чувствительности, сформированный итоговым сценарием

Этот график строится по данным `sir_scan.csv` и показывает зависимость `peak_I` от параметра β . Он дублирует по смыслу уже рассмотренную зависимость из `sir_scan.png`, но в итоговом сценарии сохраняется отдельно как отчётный рисунок `sensitivity.png`.

5.8.4 Производные форматы итогового сценария

```
julia> Literate.script("scripts/sirpetri_report.jl", "scripts"; name="sirpetri_report_clean")
[ Info: generating plain script file from `~/imit-model/2026-1--study--simulation-modeling/labs/lab06/lab_06_SIR_petri/scripts/sirpetri_report.jl`
[ Info: writing result to `~/imit-model/2026-1--study--simulation-modeling/labs/lab06/lab_06_SIR_petri/scripts/sirpetri_report_clean.jl`
"/home/daidrisov/imit-model/2026-1--study--simulation-modeling/labs/lab06/lab_06_SIR_petri/scripts/sirpetri_report_clean.jl"
```

Clean-версия итогового отчётного сценария

`Clean`-файл содержит исполняемый вариант итогового сценария без literate-комментариев.

```
julia> Literate.notebook("scripts/sirpetri_report.jl", "notebook"; name="sirpetri_report", execute=true)
[ Info: generating notebook from `~/imit-model/2026-1--study--simulation-modeling/labs/lab06/lab_06_SIR_petri/scripts/sirpetri_report.jl`
[ Info: executing notebook `sirpetri_report.ipynb`
[ Info: writing result to `~/imit-model/2026-1--study--simulation-modeling/labs/lab06/lab_06_SIR_petri/notebook/sirpetri_report.ipynb`
"/home/daidrisov/imit-model/2026-1--study--simulation-modeling/labs/lab06/lab_06_SIR_petri/notebook/sirpetri_report.ipynb"
```

Notebook-представление итогового сценария

Notebook-версия удобна для повторного чтения CSV-файлов и построения итоговых графиков в интерактивном режиме.

```
julia> Literate.markdown("scripts/sirpetri_report.jl", "docs"; name="sirpetri_report", execute=true)
[ Info: generating markdown page from `~/imit-model/2026-1--study--simulation-modeling/labs/lab06/lab_06_SIR_petri/scripts/sirpetri_report.jl`
[ Info: writing result to `~/imit-model/2026-1--study--simulation-modeling/labs/lab06/lab_06_SIR_petri/docs/sirpetri_report.md`
"/home/daidrisov/imit-model/2026-1--study--simulation-modeling/labs/lab06/lab_06_SIR_petri/docs/sirpetri_report.md"
```

Markdown-представление итогового сценария

Markdown-файл завершает цепочку производных артефактов и документирует назначение итогового отчётного сценария.

6 Выводы

В ходе выполнения лабораторной работы была реализована модель SIR в аппарате сетей Петри, подготовлен модуль `SIRPetri.jl` и набор сценариев для базового прогона, параметрического исследования, анимации и итогового анализа. Для базового эксперимента были получены две траектории: гладкая детерминированная и событийная стохастическая.

Все три CSV-файла и все графики были проанализированы. Детерминированная модель дала пик $I \approx 952.69$ уже при $t = 0.5$, стохастическая модель достигла $I = 999$ при $t \approx 0.037$, а параметрическое исследование показало слабую зависимость `peak_I` и `final_R` от β в текущей постановке. Для каждого сценария были успешно подготовлены `clean`, `md` и `ipynb` представления, а затем материалы интегрированы в отчёт и презентацию.

Список литературы

1. Peterson J. L. Petri Net Theory and the Modeling of Systems. Englewood Cliffs, NJ: Prentice Hall, 1981.
2. Gillespie D. T. [Exact Stochastic Simulation of Coupled Chemical Reactions](#) // The Journal of Physical Chemistry. 1977. Т. 81, № 25. С. 2340–2361.
3. Bezanson J. и др. [Julia: A Fresh Approach to Numerical Computing](#) // SIAM Review. 2017. Т. 59, № 1. С. 65–98.
4. Datseris G., contributors. [DrWatson.jl Documentation](#). 2024.
5. Ekre F., contributors. [Literate.jl Documentation](#). 2024.
6. Knuth D. E. [Literate Programming](#) // The Computer Journal. 1984. Т. 27, № 2. С. 97–111.
7. Schulte E. и др. [A Multi-Language Computing Environment for Literate Programming and Reproducible Research](#) // Journal of Statistical Software. 2012. Т. 46, № 3. С. 1–24.

Повторное использование

[CC BY 4.0](#)